Week 2 : DATA STRUCTURES AND ALGORITHMS

- Linear Search Time Complexity:
  - Best Case: O(1)
  - Average Case: O(n)
  - Worst Case: O(n)
- Binary Search Time Complexity:
  - Best Case: O(1)
  - Average Case: O(log n)
  - Worst Case: O(log n)

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
   - Explain Big O notation and how it helps in analyzing algorithms.
   - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
   - Create a class **Product** with attributes for searching, such as **productId, productName**, and **category**.
3. **Implementation:**
   - Implement linear search and binary search algorithms.
   - Store products in an array for linear search and a sorted array for binary search.
4. **Analysis:**
   - Compare the time complexity of linear and binary search algorithms.

- Discuss which algorithm is more suitable for your platform and why.

CODE :

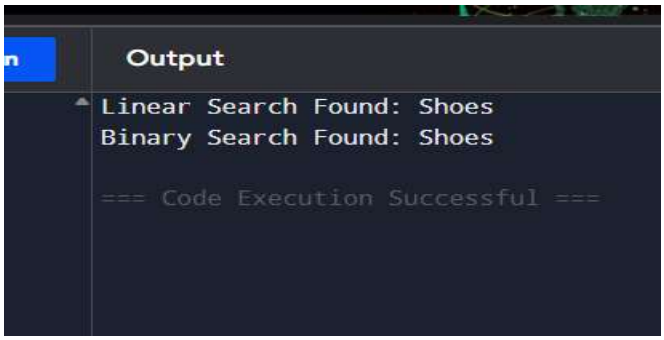
Main.cs

```
using System;
class Product
{
    public int ProductId;
    public string ProductName;
    public string Category;

    public Product(int id, string name, string category)
    {
        ProductId = id;
        ProductName = name;
        Category = category;
    }
}
class Program
{
    static Product LinearSearch(Product[] products, int id)
    {
        foreach (Product p in products)
        {
            if (p.ProductId == id)
                return p;
        }
        return null;
    }
```

```
    // Binary Search
    static Product BinarySearch(Product[] products, int id)
    {
        int left = 0, right = products.Length - 1;
        while (left <= right)
        {
            int mid = (left + right) / 2;
            if (products[mid].ProductId == id)
                return products[mid];

            if (products[mid].ProductId < id)
                left = mid + 1;
            else
                right = mid - 1;
        }

        return null;
    }

    static void Main()
    {
        Product[] products =
        {
            new Product(101,"Laptop","Electronics"),
            new Product(102,"Phone","Electronics"),
```

```
    static void Main()
    {
        Product[] products =
        {
            new Product(101,"Laptop","Electronics"),
            new Product(102,"Phone","Electronics"),
            new Product(103,"Shoes","Fashion"),
            new Product(104,"Watch","Accessories"),
            new Product(105,"Book","Education")
        };

        int searchId = 103;

        Product result1 = LinearSearch(products, searchId);

        if (result1 != null)
            Console.WriteLine("Linear Search Found: " + result1.ProductName);

        Product result2 = BinarySearch(products, searchId);

        if (result2 != null)
            Console.WriteLine("Binary Search Found: " + result2.ProductName);
    }
}
```

OUTPUT :



Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**

- **Explain the concept of recursion and how it can simplify certain problems.**

2. **Setup:**
   - **Create a method to calculate the future value using a recursive approach.**
3. **Implementation:**
   - **Implement a recursive algorithm to predict future values based on past growth rates.**
4. **Analysis:**
   - **Discuss the time complexity of your recursive algorithm.**

   Explain how to optimize the recursive solution to avoid excessive computation.

Analysis

- Time Complexity: **O(n)**
- Space Complexity: **O(n)** (due to recursive call stack)

CODE :

Online Compiler

```
using System;
class Program
{
    static double Forecast(double currentValue, double growthRate, int years)
    {
        if (years == 0)
            return currentValue;
        return Forecast(currentValue, growthRate, years - 1)
                * (1 + growthRate);
    }
static void Main(string[] args)
    {
        double presentValue = 10000;
        double growthRate = 0.10; // 10%
        int years = 5;

        double futureValue = Forecast(presentValue, growthRate, years);

        Console.WriteLine("Present Value : " + presentValue);
        Console.WriteLine("Growth Rate   : " + (growthRate * 100) + "%");
        Console.WriteLine("Years         : " + years);
        Console.WriteLine("Future Value  : " + futureValue);
    }
}
```